

A Technical Introduction to Markov decision process

Section 1

If there exists a function h , then $V^* = \bar{V}$ will be the smallest g satisfying the above equation. In order to find V^* , we could use the following linear programming model:

Section 2

In addition, transition probability is sometimes written $\Pr(s, a, s')$, $\Pr(s' \mid s, a)$ or, rarely, $p_{s's}(a)$.

Section 3

The solution above assumes that the state s is known when action is to be taken; otherwise $\pi(s)$ cannot be calculated. When this assumption is not true, the problem is called a partially observable Markov decision process or POMDP.

Section 4

Value iteration In value iteration (Bellman 1957), which is also called backward induction, the π function is not used; instead, the value of $\pi(s)$ is calculated within $V(s)$ whenever it is needed. Substituting the calculation of $\pi(s)$ into the calculation of $V(s)$ gives the combined step;

Section 5

Discrete space: Linear programming formulation If the state space and action space are finite, we could use linear programming to find the optimal policy, which was one of the earliest approaches applied. Here we only consider the ergodic model, which means our continuous-time MDP becomes an ergodic continuous-time Markov chain under a stationary policy. Under this assumption, although the decision maker can make a decision at any time in the current state, there is no benefit in taking multiple actions. It is better to take an action only at the time when system is transitioning from the current state to another state. Under some conditions, if our optimal value function V^* is independent of state i , we will have the following inequality:

Section 6

for all feasible solution $y(i, a)$ to the D-LP. Once we have found the optimal solution $y^*(i, a)$, we can use it to establish the optimal policies.

Section 7

$y(i, a)$ is a feasible solution to the D-LP if $y(i, a)$ is nonnegative and satisfied the constraints in the D-LP problem. A feasible solution $y^*(i, a)$ to the D-LP is said to be an optimal solution if

Section 8

A is a set of actions called the action space (alternatively, A_s is the set of actions available from state s). As for state, this set may be discrete or continuous.

Section 9

We could solve the equation to find the optimal value function V^* , which in turns yield the optimal control at any time t , $a(t)$ through

Section 10

$g \geq R(i, a) + \sum_{j \in S} q(j \mid i, a) h(j) \forall i \in S \text{ and } a \in A(i)$

Section 11

$Q(s, a) = \sum_{s'} P_{a(s, s')} (R_{a(s, s')} + \gamma V(s'))$

Section 12

$\pi(s) := \operatorname{argmax}_a \{ \sum_{s'} P_{a(s, s')} (R_{a(s, s')} + \gamma V(s')) \}$

Section 13

$V(s) := \sum_{s'} P_{\pi(s)(s, s')} (R_{\pi(s)(s, s')} + \gamma V(s'))$

Section 14

Category theoretic interpretation Other than the rewards, a Markov decision process (S, A, P) can be understood in terms of Category theory. Namely, let A denote the free monoid with generating set A . Let $\mathbf{Dist}$ denote the Kleisli category of the Giry monad. Then a functor $A \rightarrow \mathbf{Dist}$ encodes both the set S of states and the probability function P . In this way, Markov decision processes could be generalized from monoids (categories with one object) to arbitrary categories. One can call the result $(C, F: C \rightarrow \mathbf{Dist})$ a context-dependent Markov decision process, because moving from one object to another in C

$\mathcal{C}$ changes the set of available actions and the set of possible states.

Section 15

Computational complexity Algorithms for finding optimal policies with time complexity polynomial in the size of the problem representation exist for finite MDPs. Thus, decision problems based on MDPs are in computational complexity class P. However, due to the curse of dimensionality, the size of the problem representation is often exponential in the number of state and action variables, limiting exact solution techniques to problems that have a compact representation. In practice, online planning techniques such as Monte Carlo tree search can find useful solutions in larger problems, and, in theory, it is possible to construct online planning algorithms that can find an arbitrarily near-optimal policy with no computational complexity dependence on the size of the state space.

Section 16

$P_a(s, s')$ is, on an intuitive level, the probability that action a in state s at time t will lead to state s' at time $t + 1$. In general, this probability transition is defined to satisfy $\Pr(s_{t+1} \in S' \mid s_t = s, a_t = a) = \int_{S'} P_a(s, s') ds'$, for every $S' \subseteq S$ measurable. In case the state space is discrete, the integral is intended with respect to the counting measure, so that the latter simplifies as $P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$; in case $S \subseteq \mathbb{R}^d$, the integral is usually intended with respect to the Lebesgue measure.